

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Oberst Eduard, FAF-243

Report

Laboratory work nr. 7

Three Small Programs: Processes, Redirection, a Pipe

Course: Operating Systems (Track A: Linux)

Checked by:
Patricia Reitman, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Purpose of the work

The lecture left three questions open: how a process is born, where `printf` actually writes, and how `ls | wc -l` works. The purpose of this laboratory work is to answer all three by running them rather than by reading about them: to build `run.c`, which performs by hand the three steps a shell takes for `ls -l` — `fork`, `exec` and `wait` — and to make its child become a different program and then a program that does not exist; to create a zombie on purpose in `zombie.c` and watch it in the process table; to write `redirect.c`, which does what the shell does for `ls -l > out.txt` with nothing but `close` and `open`; and to build `ls /etc | wc -l` from one `pipe` and two `fork`s in `pipeline.c`, breaking it afterwards on purpose to see what a pipe really waits for. The optional three-stage pipeline `pipeline3.c` was written as well.

All work was done in `~/OS_26/lab7` on Ubuntu 26.04.1 LTS, kernel 7.0.0-31-generic, with gcc 15.2.0. Every program is a single `.c` file compiled with `gcc -Wall -Wextra`, and all five compile without a single warning. The experiments that required editing a program — changing the `execlp` line in Part 1 and commenting out a `close` in Part 4 — were done in place and then reverted, so the submitted sources are the versions the lab specifies; the variants written to support an answer were compiled from copies and are named in the text. In the figures the host prompt is shortened to `$`, and `ps` output is filtered to the processes of the experiment, since the full table on a desktop session is several hundred lines long. The complete session is in `lab7-transcript.log`.

Preparation

```
$ mkdir -p ~/OS_26/lab7 && cd ~/OS_26/lab7
$ gcc --version | head -1
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
$ lsb_release -ds ; uname -sr
Ubuntu 26.04.1 LTS
Linux 7.0.0-31-generic
$ make
gcc -Wall -Wextra -o run run.c
gcc -Wall -Wextra -o zombie zombie.c
gcc -Wall -Wextra -o redirect redirect.c
gcc -Wall -Wextra -o pipeline pipeline.c
gcc -Wall -Wextra -o pipeline3 pipeline3.c
```

Figure 0.1 – The toolchain, and all five programs compiling without warnings

A small `Makefile` with the single pattern rule `%: %.c` and `CFLAGS = -Wall -Wextra` was added so that the whole set rebuilds with one command; it is the only file in the folder that the lab did not ask for.

Part 1. fork + exec + wait

`run.c` performs the three steps the shell takes for `ls -l`. `fork` copies the process, the copy calls `execlp` and becomes `ls`, and the original blocks in `waitpid` until the copy is finished. The two `fflush(stdout)` calls are not decoration — the end of this part shows what the program prints without them.

Interesting Command Output:

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 130657
child:  my PID is 130658, my parent is 130657
total 112
-rw-rw-r-- 1 capziro capziro   184 Oct 19 14:09 Makefile
-rwxrwxr-x 1 capziro capziro 16296 Oct 19 14:09 pipeline
-rw-rw-r-- 1 capziro capziro  1014 Oct 19 14:09 pipeline.c
-rwxrwxr-x 1 capziro capziro 16296 Oct 19 14:09 pipeline3
-rw-rw-r-- 1 capziro capziro  1440 Oct 19 14:09 pipeline3.c
-rwxrwxr-x 1 capziro capziro 16336 Oct 19 14:09 redirect
-rw-rw-r-- 1 capziro capziro   706 Oct 19 14:09 redirect.c
-rwxrwxr-x 1 capziro capziro 16392 Oct 19 14:09 run
-rw-rw-r-- 1 capziro capziro   912 Oct 19 14:09 run.c
-rwxrwxr-x 1 capziro capziro 16432 Oct 19 14:09 zombie
-rw-rw-r-- 1 capziro capziro   980 Oct 19 14:09 zombie.c
parent: child 130658 ended with status 0
```

Figure 1.1 – The child becomes ls -l; the order of the lines is the point

```
$ grep -n execlp run.c
19:        execlp("date", "date", (char *)NULL);    /* step 2: become date */
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 130679
child:  my PID is 130680, my parent is 130679
Mon Oct 19 02:09:33 PM EEST 2026
parent: child 130680 ended with status 0
```

Figure 1.2 – Step 1.3: the same child becomes date instead

```
$ grep -n execlp run.c
19:        execlp("nosuchprog", "nosuchprog", (char *)NULL);
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 130688
child:  my PID is 130689, my parent is 130688
exec: No such file or directory
parent: child 130689 ended with status 127
```

Figure 1.3 – Step 1.4: a program that does not exist

The three runs differ in one line of source and in nothing else, which makes the comparison in Table 1.1 exact.

Table 1.1 – The same program with three different execlp lines

execlp asks for	What reached the screen	Status	Who produced the status
`ls`	the directory listing	0	`ls` itself, having succeeded
`date`	the date and time	0	`date` itself, having succeeded
`nosuchprog`	`exec: No such file or directory`	127	`exit(127)` in our own source

Observations:

1. **The line `perror("exec")` is in the child, right after `execlp`. Why is it printed only with `nosuchprog`?**

   Because a successful `execlp` never returns. It does not call the new program, it replaces the calling one: the kernel discards the old text, data, heap and stack and loads `ls` in their place, keeping the same PID, the same parent and the same open descriptors. The instruction after `execlp` belongs to a program image that no longer exists, so there is nothing left to execute it. `execlp` returns only when the replacement could not happen, and then it returns -1 with `errno` set. Reaching `perror("exec")` is therefore itself the error report, and the message it printed in Figure 1.3 is the `errno` the kernel chose: `ENOENT`, "No such file or directory", because there is no `nosuchprog` on any directory of `PATH`. The same asymmetry explains the two ways these runs ended: `ls` and `date` left through their own `exit`, while the failed run fell through to the `exit(127)` on the line below `perror`.

2. **Which number does the child print as "my parent", and whose PID is it?**

   In Figure 1.1 the child prints 130657, which is exactly the number the parent printed one line earlier with `getpid()`. So `getppid()` in the child returns the PID of the `./run` process that called `fork` — the child's parent is the program itself, not the shell. That is the point of the pair of calls: `fork` returns twice, 0 in the child and the child's PID in the parent, so each process can name the other from its own side, the parent through the return value and the child through `getppid()`. The number is a property of the run and not of the program: it was 130679 and 130688 in the next two runs, because each `./run` is a new process with a new PID.

3. **What status does the parent see with `ls`, and what status with `nosuchprog`?**

   `WEXITSTATUS(status)` is **0** for `ls` and **127** for `nosuchprog`. The 0 belongs to `ls`: it listed the directory successfully and exited 0, and that is the byte `waitpid` collected out of the child's process table entry. The 127 belongs to no program that ran, because nothing ran — it is the literal `exit(127)` in our own source, reached because `execlp` came back. The value is not arbitrary either: 127 is the conventional "command not found" status, the same one `bash` reports for a misspelled command, so a shell built on top of this code would already agree with the convention without any extra work. It is worth noting that `status` is not the number itself but a packed word, which is why `WEXITSTATUS` has to extract it.

Extra: what the two fflush(stdout) calls are for

```
$ grep -c fflush run_noflush.c          # both fflush lines removed
0
$ ./run_noflush | cat
total 116
-rw-rw-r-- 1 capziro capziro   184 Oct 19 14:09 Makefile
-rw-r--r-- 1 capziro capziro     0 Oct 19 14:10 out.txt
   ... the rest of the listing ...
-rw-rw-r-- 1 capziro capziro   980 Oct 19 14:09 zombie.c
parent: my PID is 131355
parent: child 131357 ended with status 0


$ ./run_noflush                         # the same binary, straight to a terminal
parent: my PID is 131240
child:  my PID is 131241, my parent is 131240
total 116
   ...
```

Figure 1.4 – The same program without fflush: wrong into a pipe, right on a terminal

Two things go wrong at once, and both are invisible on a terminal. The parent's two messages moved to the **end** of the output, because `stdout` connected to a pipe is block-buffered instead of line-buffered, so they sat in the buffer until `exit` flushed it — long after `ls` had written its listing straight to the descriptor. And the child's message disappeared **completely**: it was sitting in the child's own copy of that buffer, and `execlp` threw the buffer away with the rest of the address space before anything flushed it. The reason both happen is that the buffer is ordinary user-space memory belonging to the C library, not kernel state: `fork` duplicates it and `exec` destroys it, while the descriptor it would eventually be written to survives both. Run straight to a terminal

the program looks perfectly correct, which is the strongest argument for having tested it through a pipe.

Part 2. A zombie, on purpose

A zombie is a child that has ended but has not yet been waited for. `zombie.c` is `run.c` with one line inserted, so that the interval between the child's death and the parent's `waitpid` lasts long enough to be photographed with `ps`.

Interesting Command Output:

```
    int status;
    sleep(20);                                  /* the new line */
    waitpid(pid, &status, 0);                   /* step 3: wait for the child */
```

Figure 2.1 – The only difference between zombie.c and run.c

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
 131086  131083 S    /bin/bash part2c.sh
 131087  131086 S    ./zombie
 131089  131087 Z    [ls] <defunct>


   ... 20 s later, the same command and the same PIDs ...


$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
 131086  131083 S    /bin/bash part2c.sh
```

Figure 2.2 – Steps 2.2 and 2.3: the zombie, and its absence twenty seconds later

```
$ grep -E '^(Name|State)' /proc/130874/status
Name:   ls
State:  Z (zombie)
$ cat /proc/130874/cmdline | wc -c
0
```

Figure 2.3 – The same process in the kernel's own words, with nothing left of it

Table 2.1 – The STAT letters that appear in Figure 2.2

STAT	Name	What it means for the process in this figure
`S`	interruptible sleep	The parent `./zombie`, blocked inside `sleep(20)`.
`Z`	zombie / defunct	The child `ls`: finished, status kept, not yet waited for.
`Ss`	sleeping, session leader	The shell that started the job; the trailing `s` is a flag, not a state.

Observations:

1. **Which letter does `ls` have in the STAT column, and what does it mean?**

   Z, which `ps` spells out in the CMD column as `[ls] <defunct>` and which `/proc/<pid>/status` writes in full as `State: Z (zombie)`. It means that `ls` has already run to completion and called `exit`: its memory, its open descriptors and its kernel stack are all gone. Figure 2.3 is the proof — `/proc/130874/cmdline` is 0 bytes long, because the command line lived in the address space that was torn down, and this is also why `ps` has to print the name in square brackets, from the last thing it remembers rather than from the process. What survives is a single entry in the process table holding the PID and the exit status, kept because somebody may still ask for it. A zombie therefore costs no CPU and no memory, only a PID — but PIDs are a finite resource, which is why a long-running program that forgets to wait for its children eventually cannot fork at all.

2. **After 20 seconds, is the `[ls] <defunct>` line still there? Which line of `zombie.c` made it disappear?**

   No — it is gone, and `./zombie` itself is gone with it, as the second half of Figure 2.2 shows for the very same PIDs. The line responsible is `waitpid(pid, &status, 0);`. That call is precisely what the dead child was waiting for: it copies the saved exit status into `status`, and the kernel then releases the process table entry, which is what "reaping" means. The parent vanishes in the same instant for an unrelated reason — it prints its last message and returns. The experiment shows that a zombie is not a malfunction but a deliberate interval: the exit status has to be preserved from the moment the child exits until the moment its parent asks for it, and all `sleep(20)` does is stretch that interval from microseconds to something `ps` can catch.

Extra: what if the parent never waits at all?

```
$ ps -o pid,ppid,stat,cmd -p 130961 -p 130963
    PID    PPID STAT CMD
 130961  130862 S    ./zombie
 130963  130961 Z    [ls] <defunct>
$ kill -9 130961         # the parent dies without ever calling waitpid
[1]+  Killed                 ./zombie > /dev/null
$ ps -o pid,ppid,stat,cmd -p 130961 -p 130963
    PID    PPID STAT CMD
(neither process exists any more)
```

Figure 2.4 – Killing the negligent parent: the zombie goes too

Killing the parent in the middle of its sleep does not leave the zombie stranded for ever. An orphaned process is re-parented to `init`/`systemd`, PID 1, and part of PID 1's job is to call `wait` in a loop, so the entry was reaped the moment its new parent noticed it. A zombie thus lives exactly as long as the parent that is failing to wait for it, which is why zombies are a symptom of a long-lived server with a missing `wait` and never of a short-lived command.

Part 3. Redirection: ls -l > out.txt

`redirect.c` does by hand what the shell does for `ls -l > out.txt`, and it does it with two calls and no cleverness: in the child it closes descriptor 1, then opens the file, which is handed the smallest free descriptor number — 1. `ls` is then started and changed in no way whatsoever.

Interesting Command Output:

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 112
-rw-rw-r-- 1 capziro capziro   184 Oct 19 14:09 Makefile
-rw-r--r-- 1 capziro capziro     0 Oct 19 14:09 out.txt
$ wc -l out.txt
13 out.txt
```

Figure 3.1 – Step 3.2: open returns 1, and the whole listing is in the file

```
$ ./redirect | cat
open gave descriptor 1
done: the listing is in out.txt
```

Figure 3.2 – Not on the terminal, and not in a pipe either: in the file

```
$ ./redirect_noclose                  # the same program with close(1) removed
open gave descriptor 3
total 116
-rw-rw-r-- 1 capziro capziro   184 Oct 19 14:09 Makefile
   ... the whole listing, on the terminal ...
-rw-rw-r-- 1 capziro capziro   980 Oct 19 14:09 zombie.c
done: the listing is in out.txt
$ head -2 out.txt                     # nothing: created, truncated, never written
```

Figure 3.3 – Without the close, open returns 3 and the redirection does not happen

Table 3.1 – The child's descriptor table, before and after the two calls

fd	After fork	After close(1)	After open("out.txt")
0	the terminal	the terminal	the terminal
1	the terminal	— free —	`out.txt` ← open returns 1
2	the terminal	the terminal	the terminal

Observations:

1. **Which number did `open` give, and why exactly that one?**

   It gave **1**, as the first line of Figure 3.1 reports. `open` is specified to return the lowest-numbered descriptor not currently in use by the calling process, and the `close(1)` on the line above had just made 1 unused while 0 and 2 stayed open — so 1 was the smallest hole in the table, as Table 3.1 sets out column by column. There is no relationship between `open` and standard output: the number is simply the first free slot, and the slot was emptied on purpose one line earlier. Figure 3.3 proves the rule by removing the `close`: with 0, 1 and 2 all still occupied the identical `open` call returned **3**, the listing went to the terminal, and `out.txt` was left empty — created and truncated by `O_CREAT | O_TRUNC`, but never written to. The same truncation explains a detail inside Figure 3.1 itself: `out.txt` appears in its own listing with a size of 0, because `open` emptied it before `ls` ran and `ls` read its size before any of the 13 lines had been written.

2. **`ls` was not changed at all. Why does its listing not appear on the screen?**

   Because `ls` writes to descriptor 1 — it always has, and it still does — and in this child descriptor 1 no longer refers to the terminal but to `out.txt`. The whole redirection was finished before `ls` existed: `fork` gave the child its own copy of the descriptor table, `close` and `open` rearranged that copy, and `execlp` then replaced the program image while leaving the table untouched, because descriptors live in the kernel and not in the address space. `ls` inherits an

environment prepared for it and cannot tell the difference; there is no flag, no argument and no code path inside `ls` that knows about redirection, which is exactly why `> out.txt` works with every program ever compiled, including ones older than the shell being used. Figure 3.2 makes the point sharper still: piping `./redirect` into `cat` does not reveal the listing either, since the child's descriptor 1 is the file in that case too. What does still reach the screen is the parent's own `printf` — the parent closed nothing, and only the child's table was altered.

3. **To redirect the errors instead (like `2> err.txt`), which descriptor would you close before `open`?**

   Descriptor **2**. It is the same mechanism with a different hole: close 2 and 2 becomes the lowest free number, so that is what `open` returns. This was verified with a variant that closes 2, opens `err.txt`, and runs `ls /nosuchdir` so that there is a diagnostic to capture:

```
$ ./redirect_err
open gave descriptor 2
done: the error message is in err.txt
$ cat err.txt
ls: cannot access '/nosuchdir': No such file or directory
```

Figure 3.4 – close(2) instead of close(1): the errors are redirected, 2> err.txt

`open` returned 2, the error message landed in the file, and the terminal stayed clean while the normal output of `ls` would still have gone to it — which is the entire difference between `>` and `2>`. Writing this variant also reproduced the trap from Part 1: the first attempt printed the descriptor number with `printf` and no `fflush`, and the line never appeared at all, because `exec` discarded the buffer holding it. The message only showed up after `fflush(stdout)` was added — the same lesson as Figure 1.4, met by accident in a different part of the lab.

Part 4. A pipe: ls /etc | wc -l

`pipeline.c` builds the pipeline by hand out of one `pipe` and two `fork`s. In the `ls` child descriptor 1 is closed and `dup(p[1])` copies the writing end into the hole that leaves; in the `wc` child descriptor 0 is closed and `dup(p[0])` copies the reading end into that hole. `dup` obeys the same lowest-free-first rule as `open` in Part 3, which is what makes the construction work. Everyone then closes both original ends — including the parent, which uses neither.

Interesting Command Output:

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
247
$ ls /etc | wc -l
247


$ gcc -Wall -Wextra -o pipeline3 pipeline3.c && ./pipeline3
243
$ ls /usr/bin | grep x | wc -l
243
```

Figure 4.1 – Step 4.2 and the optional part: both agree with the shell

Table 4.1 – Where each descriptor points after the close and the dup

Process	**closes**	**then dup gives it**	**so the program sees**
`ls`	`1`	`1` → `p[1]`	stdout is the pipe
`wc`	`0`	`0` → `p[0]`	stdin is the pipe
parent	`p[0]`, `p[1]`	— nothing —	it is not in the pipeline at all

Step 4.3: breaking it on purpose

```
$ sed -n '33,35p' pipeline.c
    close(p[0]);                                /* the parent uses neither end */
    /* close(p[1]); */
    wait(NULL);
$ gcc -Wall -Wextra -o pipeline pipeline.c
$ ./pipeline            # never ends; stopped after 5 s instead of Ctrl-C
exit status: 124        (124 = killed by timeout, i.e. still running)
```

Figure 4.2 – With the parent's close(p[1]) commented out, nothing is printed at all

```
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
 131122  131120 S    ./pipeline_broken
 131125  131122 S    wc -l
$ for c in $(ps -o pid= --ppid 131122); do >     echo "pid $c comm=$(cat /proc/$c/comm) wchan=$
(cat /proc/$c/wchan)"; done
pid 131125 comm=wc state=S wchan=anon_pipe_read
```

Figure 4.3 – Only wc is left, asleep in the kernel's pipe-read wait

Observations:

1. **Did `./pipeline` and `ls /etc | wc -l` give the same number?**

   Yes — **247** both times, as Figure 4.1 shows side by side. They agree because they are the same construction: one `pipe`, two `fork`s, a `close` and a `dup` in each child to move the right end of the pipe onto the right standard descriptor, and a parent that closes both ends and waits twice. The optional three-stage version agrees as well, printing **243** exactly like `ls /usr/bin | grep x | wc -l`; it needs

two pipes and three children, and the rule that makes it work is that every child closes all four pipe ends after its `dup`s and the parent closes all four as well. Nothing in either program is specific to the commands chosen — `execlp` is the only line that names them.

2. **After the change, which of the two children never ends, `ls` or `wc`?**

   `wc`. Figure 4.3 lists exactly one surviving child, `wc -l`, and `ls` is not in the table at all: it wrote its 247 lines, exited normally, and the parent's first `wait(NULL)` reaped it. The parent is now blocked in its second `wait(NULL)` for a child that will never exit, which is why the program as a whole hangs and had to be stopped. Figure 4.2 shows a second consequence that is easy to overlook: the broken run prints **nothing at all**, not a wrong number, because `wc -l` emits its count only once it has finished counting — and it never finishes.

3. **Why, in one sentence: what is `wc` waiting for?**

   `wc` is waiting for end-of-file on its descriptor 0, and a pipe reports end-of-file only when the **last** writing end is closed — the parent still holds `p[1]` open, so `read` has no reason to return 0 and blocks for ever. The kernel names the wait itself in Figure 4.3: `/proc/131125/wchan` reads `anon_pipe_read`. What the pipe counts is writers, not writes, and the parent is a writer that never intends to write a single byte — the pipe cannot tell the difference between that and a writer which is merely slow. This is the reason every process in a pipeline must close every end it does not use, and the reason that duty falls on the parent just as much as on the children.

Conclusion

Three programs of about thirty lines each answered the three questions the lecture had left open, and in every case the answer turned out to be that the shell has no special powers at all — it calls the same handful of system calls that any C program may call.

A process is born by `fork`, which returns twice, and becomes a different program by `exec`, which returns once and only on failure. That asymmetry is the whole of Part 1: `perror("exec")` appeared only in the `nosuchprog` run, because in the two successful runs there was no program image left to execute that line, and the status 127 the parent collected came from the `exit(127)` in our own source rather than from anything that ran, while the 0 after `ls` and `date` came from those programs themselves. `wait`

proved to be the third step and not an optional one. Putting `sleep(20)` in front of `waitpid` held the finished `ls` in state `Z` long enough for `ps` to catch it, and `/proc` confirmed how little was left of it: `State: Z (zombie)` and a `cmdline` of zero bytes, a PID and an exit status and nothing else. The single line `waitpid(pid, &status, 0)` made it disappear, and killing the parent instead showed the safety net — `init` adopts the orphan and reaps it at once. A zombie is therefore a measurable interval rather than a leak, and it becomes a leak only in a parent that lives on without ever asking.

Redirection and pipes then turned out to be the same trick applied twice, and the trick is that descriptor numbers are allocated lowest-free-first while `exec` leaves the descriptor table alone. In `redirect.c` the `close(1)` punched the hole that made `open` return 1, and removing that one line was decisive: the identical `open` returned 3 and the listing went straight back to the terminal. `ls`, unchanged and unaware, wrote into `out.txt` for the simple reason that writing to descriptor 1 is all it has ever done, and closing 2 instead of 1 captured the error stream — the entire difference between `>` and `2>`. `pipeline.c` applied the same `close`-then-`dup` to both sides of one pipe and reproduced the shell's 247 exactly, and `pipeline3.c` scaled it to two pipes, three children and 243. The deliberate bug was the most instructive moment of the lab: leaving the parent's `p[1]` open left `wc` asleep in `anon_pipe_read` for ever, because a pipe signals end-of-file by counting writers rather than writes, and the parent was a writer that never meant to write.

The through-line is that `fork`, `exec`, `wait`, `close`, `open`, `pipe` and `dup` are not machinery hidden underneath the shell — they are the shell, and a command line like `ls -l > out.txt` or `ls /etc | wc -l` is a short program written in them. The one thing that genuinely does not survive `fork` and `exec` is the C library's own output buffer, which is user-space memory rather than kernel state: without `fflush` the parent's messages were reordered to the end of a piped run and the child's were destroyed outright, twice, in two different parts of the lab. On a terminal that bug is invisible, which is the best argument for having looked through the pipe.